real-tr-p2-full

An AgentMPI run analysis

Abstract

Two agent ranks translated the eight-section book by the full stencil decomposition — `scatterv`, per-unit terminology extraction, a `UNION allreduce` over the glossary, translation, a `halo_exchange` on a two-node Cartesian line, and `gather` — in 410.11 s with no failures, no contract violations and no retries. It is the first point in the strong-scaling series at which the protocol does anything at all, and its value is that the protocol is small enough here to be read line by line: eight messages, three synchronisation points, and one seam. Speedup over the 576.74 s sequential baseline is 1.406× at efficiency 0.703, and the Karp–Flatt figure of 0.422 is the highest in the series. The document's two substantive findings are both about the cost of that single seam. First, the `halo_exchange` blocked rank 0 for 48.766 s to move 36 tokens, and none of that time appears in `coordination_share`, which reads 4.1% where the trace supports 10.0%: the halo events carry no `wall_s` and `coordination_is_underreported` does not catch the omission. Second, the two revision calls the seam triggered cost 108.33 rank-seconds of model time and changed nothing — diffing the pre- and post-revision blobs gives 0 characters moved out of 2,437, while the harness reports `n_boundary_changes: 7`, because the agents used the `changes` field to list checks they performed rather than edits they made. Seam reconciliation at $p = 2$ cost 19.2% of the run's rank-seconds and altered not one character of the book.

1 What this run is

property	value
run	`real-tr-p2-full`
experiment	`translation`
campaign label	`real-tr`
declared world size	2
ranks appearing in the log	10
executors	`broker` × 2, `worker` × 33
events	149
wall time	410.11 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.63×	total busy time over wall time
parallel efficiency	81.7%	achieved parallelism per rank
idle fraction	18.3%	rank-seconds paid for and unused
peak ranks busy	2	of 2 in the world
serial fraction of active time	34.1%	time with exactly one rank busy
load imbalance	1.10×	slowest rank's busy time over the mean
coordination share	4.1%	rank-seconds blocked in collectives, of those available
coordination in flight	8.1%	wall clock with any rank inside a collective
rank reattachments	25	fresh agent processes that took over a rank role
agent calls	18	invocations that reached a model
agent latency p50 / p95	43.52 / 62.82 s	per invocation
messages	8	6 eager, 2 rendezvous
tokens sent	7,978	5,321 deferred under rendezvous
tokens in / out	32,289 / 5,964	prompt and completion
cost	$0.1863	0.0312 $/kTok out

3 What the analysis notices

- [warning] `halo_exchange/?` reports 0 messages but the fabric logged 2: the collective's own accounting disagrees with the traffic it produced.
- [warning] Ranks 2, 3, 4, 5, 6, 7, 8, 14 appear in the log without participating; the declared world size is 2. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- [warning] The cost model's α and β here are a median fallback, not a fit: the regression returned a negative slope ($\beta = 0.1427$ s/token) and was rejected. That happens when queueing dominates latency, because the wait enters the measured time but not the token count, so the relationship the fit looks for is not in the data. Any latency prediction from this run's calibration is a median, not a model.
- [ok] 25 rank reattachment(s) occurred, up to incarnation 11: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	301.63	73.5	9	9	4	4	5,646	0.0	finalized
1	368.90	90.0	9	9	4	4	2,332	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

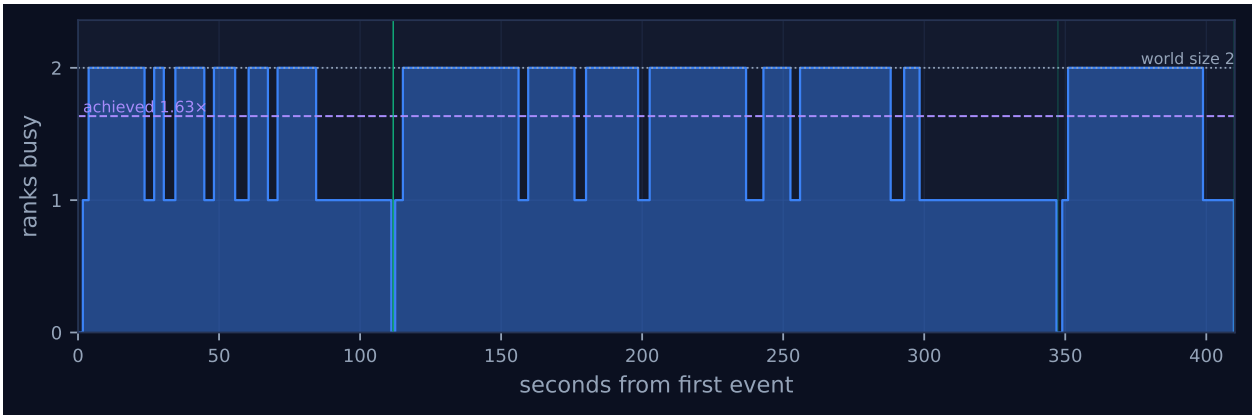

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

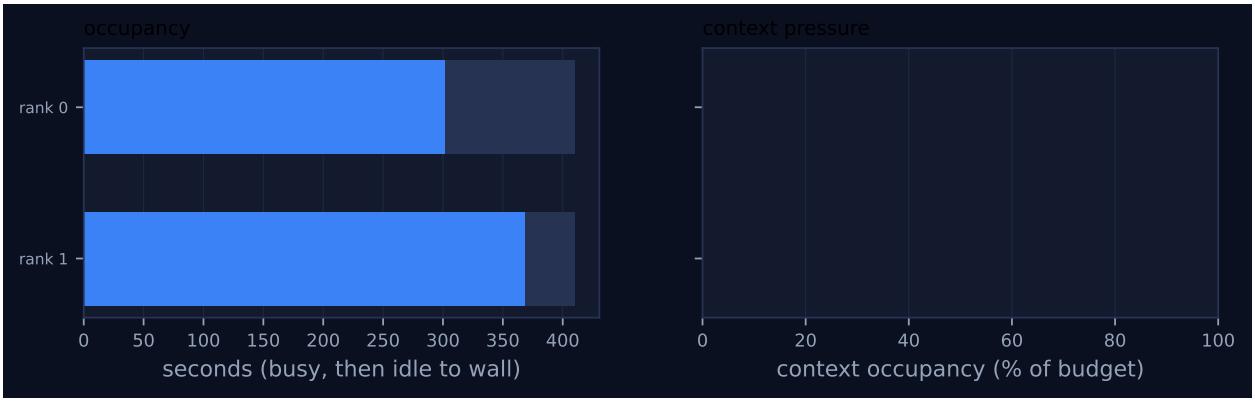

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

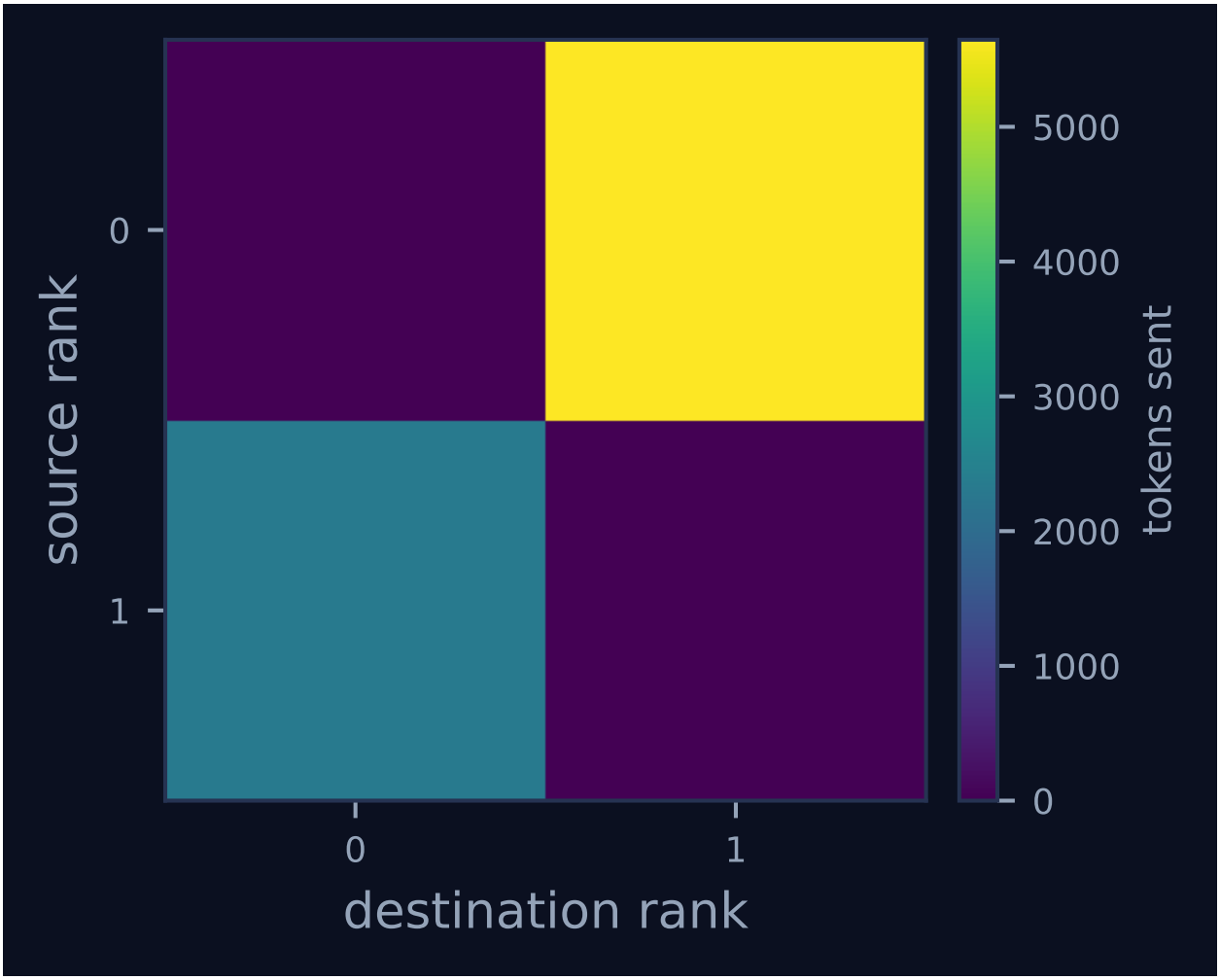

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

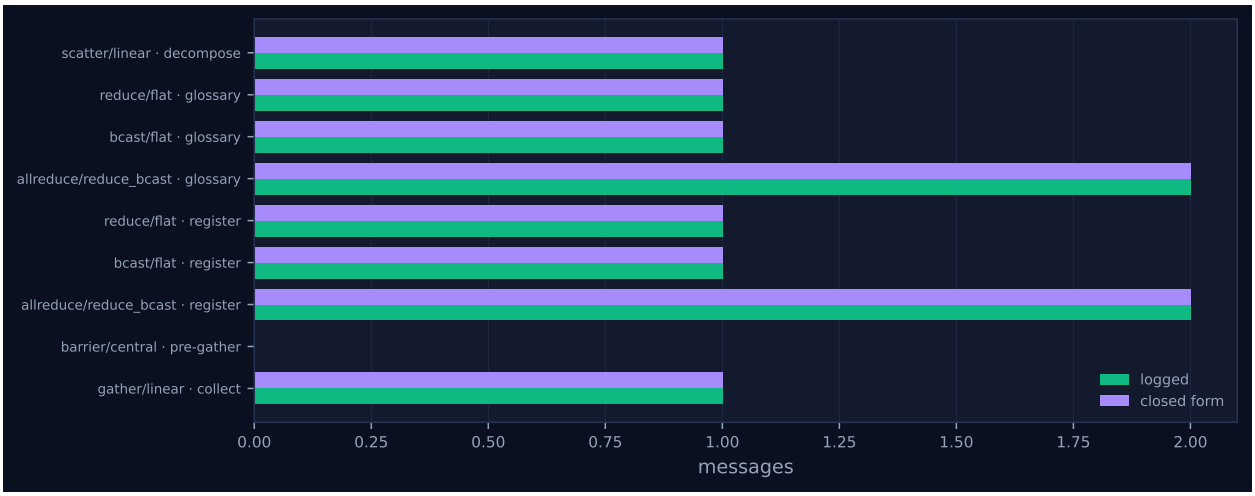

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	linear	decompose	2	2	1	1	1	1	3,911	0.01	0.01	✓
reduce	flat	glossary	2	2	1	1	1	1	845	22.24	0.24	✓
bcast	flat	glossary	2	2	1	1	1	1	1,571	0.24	0.00	✓
allreduce	reduce_bcast	glossary	2	2	2	2	2	2	0	22.25	0.00	✓
reduce	flat	register	2	2	1	1	1	1	68	0.01	0.01	✓
bcast	flat	register	2	2	1	1	1	1	137	0.01	0.00	✓
allreduce	reduce_bcast	register	2	2	2	2	2	2	0	0.01	0.00	✓
halo_exchange	?	seams	2	2	0	–	2	–	0	0.00	0.25	–
barrier	central	pre-gather	2	2	0	2	0	0	0	10.94	0.19	✓
gather	linear	collect	2	2	1	1	1	1	1,410	0.00	0.19	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is two lanes of nine bars each, and the whole run is legible from their alignment. Both ranks start together at $t = 0$, both hold four terminology bars, both hold four translation bars, both hold one revision bar, and they end together. What separates them is that rank 1 is behind at every stage: it finishes terminology at 111.56 s against rank 0's 89.56 s, translation at 347.40 against 298.89, and revision at 409.93 against 399.17. The three gaps in rank 0's lane in Figure 1 are the three times it had to wait for rank 1 to catch up, and Figure 2 shows them as three descents from two busy ranks to one. There is no interval in the run where both ranks are idle.

The first gap is the glossary. Rank 0 returns `terms:u3` at 89.555 s and enters `allreduce(local_terms, UNION)`; under `reduce_bcast` at $p = 2$ that is a `flat` reduce to rank 0 followed by a `flat` broadcast back. Rank 1 does not return `terms:u7` until 111.562 s and sends its 845-token contribution two milliseconds later. Rank 0's `coll.reduce` lands at 111.800 s, so it blocked for 22.245 s, which is the 22.24 s the collectives table reports as the reduce's wall time. The broadcast that follows is 1,571 tokens and takes 0.24 s of rank-1 time and two milliseconds of rank-0 time. Both ranks record the same broadcast digest `2f25c998b83a` and the same 207-term glossary, which is what an exact operator over a tree must give and is not a finding.

The second gap is the seam, and it is the largest single wait in the run. Rank 0 returns `translate:u3` at 298.885 s, creates its Cartesian communicator, and at 298.887 s sends its right boundary — the last paragraph of unit 3, 27 tokens — to rank 1. It then blocks. Rank 1 is still translating unit 7 and does not reach the exchange until 347.402 s, at which point it reads rank 0's message with a logged queue residency of 48.515 s, replies with the first paragraph of unit 4 (9 tokens), and proceeds. Rank 0's `coll.halo_exchange` lands at 347.653 s, so it waited 48.766 s. Two things about that number matter. It is 11.9% of the whole run spent by half the population inside a collective that moved 36 tokens, and it is set entirely by the arrival skew, not by the exchange: rank 1's own cost was 0.002 s because its neighbour's message had been sitting in its mailbox for three quarters of a minute. At $p = 2$ the neighbourhood collective and a global barrier are the same thing, since a

rank's only neighbour is the whole rest of the world, so the argument that a halo scales better than a barrier cannot be made here — it needs $p \geq 4$ before a rank has a choice of whom to wait for.

The third gap is the `pre-gather` barrier. Rank 0 finishes its revision at 399.171 s and rank 1 at 409.925 s, so rank 0 waits 10.942 s, matching the barrier's reported wall time exactly. The `gather` that follows is one rendezvous message of 1,410 tokens and completes in 0.187 s.

Partitioning every rank-second of the run by the interval it falls in — the boundaries are `broker.enqueue`, `broker.claim`, `broker.complete`, `agent.call`, `msg.recv` and `coll.*` — gives an idle budget that closes:

where the idle rank-seconds went	rank-s	share
broker dispatch, `enqueue` to `claim`	61.420	41.1%
`halo_exchange`	48.766	32.6%
glossary `allreduce`, both invocations	22.509	15.1%
`pre-gather` barrier	10.942	7.3%
completion polling	5.800	3.9%
`scatterv`, `gather` and host-side code	0.075	0.1%
total idle, of 820.228 rank-s available	149.512	100.0%

The residual against the headline's 18.25% idle fraction is 0.19 rank-s, which is the interval between rank 1's `rank.finalize` and the end of the job. The largest row is not a collective at all. Sixty-one of the 149.5 idle rank-seconds are the broker publishing a task and waiting for a worker process to take it, eighteen times, at a mean of 3.412 s.

Figure 4 is two cells: $0 \to 1$ carrying 5,646 tokens over four messages and $1 \to 0$ carrying 2,332 over four. The asymmetry is the scatter, whose single hop is 3,911 tokens — half the book. Figure 3 shows rank 0 at 73.5% occupancy against rank 1's 90.0%, which is the same fact as the three gaps. Its right-hand panel is empty: every receive in this harness is non-admitting and the `gather` passes `admit=False`, so context occupancy is 0.0% against a 120,000-token budget for both ranks.

The viewer screenshot adds one thing the plot does not. The dashboard draws ten lanes; ranks 2 through 8 and rank 14 each carry a couple of grey lifecycle ticks and no work at all, in a world declared to be of size 2. Figure 1 draws two lanes because `analyze_run` filters to participating ranks, so the plot is the better picture of the computation and the screenshot the better picture of the job.

7 Interpretation

What is true by construction

The phase structure is the harness's. `translation.py` splits eight units between two ranks, four each, and the trace matches instruction for instruction: 2 `coll.scatter`, 4 `coll.reduce`, 4 `coll.bcast`, 4 `coll.allreduce`, 2 `topo.cart_create`, 2 `coll.halo_exchange`, 2 `coll.barrier`, 2 `coll.gather`, and 18 `agent.call`. The eighteen calls decompose as eight `terms`, eight `translate` and two `revise`, and the two follow from the decomposition rather than from any choice: with four units per rank, only the last unit of rank 0 and the first unit of rank 1 abut a rank boundary, and only those two are handed neighbour context.

Nine of the ten collective invocations sent exactly the number of messages their closed form predicts, which Figure 5 shows as matched pairs. The tenth is `halo_exchange`, discussed below.

One property of this point is not by construction but is easy to mistake for it, and it matters for the series: $p = 2$ is the only completing configuration that takes the `flat` code path. `algorithms.reduce` and `algorithms.bcast` choose `"flat" if p <= 3 else "binomial"`, so the glossary at $p = 2$ is a one-round flat reduce while at $p = 4$ and $p = 8$ it is a two- and three-round binomial tree. The scaling curve is therefore not algorithmically homogeneous across its four points. At this size the distinction costs nothing — a flat and a binomial reduce over two ranks are the same single message — but a reader comparing round counts across the curve should know that the implementation switched.

The seam cost 19% of the run and changed nothing

This is the finding the run makes best, and it took the blob store rather than the event log to establish.

The seam-reconciliation phase at $p = 2$ consists of one rank boundary, one `halo_exchange`, and two `revise` calls. Its cost is 108.332 rank-seconds of model work (49.76 s for `revise:u3` on rank 0 and 58.57 s for `revise:u4` on rank 1) plus the 48.766 rank-seconds rank 0 spent blocked in the exchange: 157.10 rank-seconds in total, 19.2% of the 820.23 rank-seconds the run had available. Against the sequential baseline it is 100% extra — `real-tr-p1-full` skips the phase entirely, because the harness guards it with `cfg.halo and comm.size > 1`.

Its effect on the book is nil. I pulled the `translate:u3`, `revise:u3`, `translate:u4` and `revise:u4` results out of `runs/real-tr-p2-full/blobs`, extracted the `translation` field from each, and diffed. Both revisions returned text byte-identical to their inputs: 1,098 characters in and 1,098 out for unit 3, 1,339 in and 1,339 out for unit 4, 0 characters moved in either. The revised sections on disk are the same bytes as the unrevised translations.

The harness nevertheless records `n_boundary_changes: 3` for rank 0 and `4` for rank 1. Reading the `changes` lists the agents returned explains the gap and makes it a reporting defect rather than a mystery: the entries are descriptions of verification, of the form "checked the terminology shared with the next section against the binding glossary and made no further alteration". The `Revision` contract asks for "a short list of the boundary or terminology fixes you made" and the agents answered with a list of the boundary and terminology items they inspected. So `n_boundary_changes` counts self-reported inspections, and the only way to count edits is to diff the artefacts, which nothing in the pipeline does. On this run the two quantities differ by the maximum possible amount: seven reported, zero real.

The sibling runs put the result in context rather than contradicting it. Applying the same diff to `real-tr-p4-full` gives three of six revisions altering the text, 22 characters of 6,959; to `real-tr-p8-full`, eight of eight, 68 characters of 9,527. That ordering has a structural explanation. The book has eight units and therefore seven internal seams, and the halo exchange can only reconcile seams that fall on a rank boundary — 1 of 7 at $p = 2$, 3 of 7 at $p = 4$, 7 of 7 at $p = 8$, 0 of 7 at $p = 1$. The four seams inside rank 0's own run of units, between u_0 and u_1 and so on, were produced by four separate agent calls with no shared context and are exactly as raw as an inter-rank seam, and the mechanism never looks at them. Cost per reconciled seam is 157.1 rank-seconds here, 125.2 at $p = 4$ and 55.9 at $p = 8$, so this configuration is the least cost-effective place in the series to run the mechanism: it pays the most per seam and reaches the fewest of them.

I would not read this as evidence that the halo exchange is useless. It is evidence that at $p = 2$ it has almost nothing to do, which is what one would predict, and that the campaign has no instrument that would have noticed. Every deterministic quality metric — coverage 1.000, consistency 1.000, `n_divergent_entities` 0, rendering verification 1.000, paragraph fidelity 1.000 — is at its ceiling in this run and in every sibling, including `real-tr-p8-nohalo`, which has no seam mechanism at all.

`coordination_share` is wrong by a factor of 2.5, and the flag that should catch it does not fire

`metrics.json` reports `coordination_share: 0.0408` and `coordination_span_share: 0.081`, with `coordination_is_underreported: false`. The first two are too low and the third should be true.

`Analysis.collective_rank_seconds` sums `rank_wall_s` over the primitive collectives, and `rank_wall_s` is assembled from the `wall_s` field each `coll.*` event carries. The `coll.halo_exchange` events in this trace carry only `{label, left, right}`: no `algorithm`, no `rounds`, no `messages_sent`, no `wall_s`. The metric therefore scores the halo at zero, and 33.474 of the run's blocking rank-seconds are counted where the trace supports 82.24. The difference, 48.77, is rank 0's halo wait to the millisecond. Recomputing directly from the event log, coordination share is 10.03% rather than 4.08%.

The same arithmetic holds at the other points of the series: 200.44 reported against 264.66 measured at $p = 4$ (16.3% against 21.5%) and 219.25 against 253.88 at $p = 8$ (16.4% against 19.0%), with the shortfall equal to the halo blocking in each case. So the corrected coordination share peaks at $p = 4$ rather than rising monotonically, which is the opposite of the shape the uncorrected numbers suggest.

`coordination_is_underreported` exists precisely to warn a reader when the blocking figure is incomplete, but its implementation is `bool(self.incomplete_collectives) or self.ok is False`. It catches the $p = 16$ failure mode, where ranks die inside a collective and never record it. It cannot catch this one, where the collective completed and recorded itself but omitted its duration. I judge this a genuine defect with two parts: the instrumentation gap in `topology.halo_exchange`, which the sibling analysis of `real-tr-p8-full` reports as closed by commit `ee21d79` after this trace was taken, and the underreporting flag, which as written would still read `false` on any run whose blocking is invisible for a reason other than failure. The second half is not fixed by the first for traces already recorded, and every trace in this campaign predates the fix.

The generated finding that `halo_exchange/?` reports 0 messages against 2 logged is the visible tip of the same gap and is a real warning, correctly raised. The fabric logged two messages tagged `_ampi:halo:0:8:R` and `_ampi:halo:0:8:L`, which is $2(p - 1) = 2$ for a non-periodic line, and the cost model has no closed form registered for the operation, which is why the row shows – and Figure 5 omits it.

The decomposition is balanced in the wrong unit

`scatterv` exists so that unequal sections can be distributed without idling ranks, and the split here is as even as it could be by the measure the harness uses. Rank 0 receives units 0–3, 2,562 words; rank 1 receives units 4–7, 2,547 words. That is a load imbalance of 1.003.

The measured busy-time imbalance is 1.100, and rank 1 is the slow one at every phase: 93.42 s of terminology against 76.60, 216.91 s of translation against 175.26, 58.57 s of revision against 49.76. Word count does not explain it, and across the whole campaign it does not explain anything: pooling the 32 translation calls of the four completing runs, the correlation between a unit's word count and its call duration is $r = -0.071$.

Paragraph count does. The same pooled correlation for paragraphs is $r = +0.489$, and it is the quantity the `Translation` contract actually constrains — "produce exactly n target paragraphs, one per source paragraph". Rank 0's four units carry 9, 17, 19 and 15 paragraphs, 60 in total; rank 1's carry 43, 26, 9 and 11, 89 in total. That is a paragraph imbalance of 1.195 against a word imbalance of 1.003, and it points the right way. Unit 4 alone has 43 paragraphs, more than twice any other unit, and it is the slowest or second-slowest translation in every run of the series.

So the imbalance here is inherent to the decomposition, but not to the decomposition the harness thinks it is making. `split_units` balances words subject to a paragraph boundary constraint; the cost driver is paragraphs. This is a small, actionable observation about the harness rather than about AgentMPI: `scatterv` did exactly what it was asked, and it was asked for the wrong split. It gets worse with p, because the paragraph imbalance of the split is 1.195 at $p = 2$, 1.852 at $p = 4$ and 2.309 at $p = 8$ while the word imbalance stays between 1.003 and 1.104.

Eight strays in a world of two

The generated findings flag ranks 2, 3, 4, 5, 6, 7, 8 and 14 as registering without participating. The fabric's `ranks` table ends with ten rows for `world_size = 2`, eight of them in state `running` with leases renewed to thirty minutes past a job that has already emitted `job.finish`.

The framing offered in the campaign — a shared worker pool, and a worker registering against the wrong job — is well supported here, and this run adds a detail. The stray index set is $\{2, \ldots, 8, 14\}$; `real-tr-p4-full`'s is $\{4, \ldots, 8, 14\}$ and `real-tr-p8-full`'s is $\{8, 14\}$. In each case it is exactly the pool's index set minus the world. The pool held indices 0–8 and 14, and this run, having the smallest world of the three, caught the most of it.

Whether AgentMPI accepting these registrations is a defect is settled in the sibling analysis of `real-tr-p8-full`, which traces it to `RankRuntime.register` performing an unconditional `INSERT` without consulting either the `world_size` meta key or `comm_members`. I agree with that reading and will not repeat the argument. What this run contributes is the containment evidence at a second world size: `comm_members` holds two rows, the reduce and broadcast trees addressed only ranks 0 and 1, the barrier's `absent` list is empty, and none of the eight strays sent or received a byte. The blast radius was again nil, and the reason it was nil is again that the misdirected indices happened to fall outside the world rather than inside it.

The calibration warning names the wrong coefficient

The remaining flagged finding says that α and β here are a median fallback rather than a fit "because the regression returned a negative slope ($\beta = 0.1427$ s/token)". The warning is worth heeding — the published $\alpha = 43.517$ s and $\beta = 0.067783$ s/token are robust medians, not a model — but the reason it gives is not the one that fired. Refitting the eighteen invocations reproduces the regression exactly: $\beta = +0.142717$ s/token, a positive slope of about seven output tokens per second, with $\alpha = -6.30$ s and $R^2 = 0.612$. The guard in `cost.calibrate` is `if beta > 0 and alpha > 0`, so it was the intercept that failed, and the finding text in `scripts/analyze_run.py` hard-codes

"negative slope" for every fallback. Its accompanying explanation — broker queueing inflating latency without inflating the token count — also does not apply, because the 3.4 s dispatch wait in this run lies between `enqueue` and `claim` and the reported `latency_s` begins at the claim. What the data actually say is that these calls are generation at a near-constant rate with no measurable fixed cost, so the sensible model has $\alpha \approx 0$, and least squares over a 209–562 token range puts the intercept slightly negative. The same message with the same wrong coefficient is printed on $p = 1$ and $p = 4$; at $p = 8$ the fit is accepted, with $R^2 = 0.250$, the worst of the four.

Transport: one message chose rendezvous, one was told to

Eight messages, six eager and two rendezvous, 7,978 tokens sent. Both ranks publish an `eager_limit` of 2,048 tokens in their `rank.init`, and exactly one payload in the run exceeds it: the scatter's single hop, `mid 1`, 3,911 tokens from rank 0 to rank 1 at $t = 0.004$ s, sent under `rendezvous` by `_choose_mode`. The other rendezvous message is the `gather`, 1,410 tokens, which is rendezvous because the harness writes `mode=Mode.RENDEZVOUS, admit=False` outright and which `AUTO` would have sent eager. The largest eager message is the glossary broadcast at 1,571 tokens.

That the scatter hop is the one that crosses the limit is a property of p, not of the data. A linear scatter over two ranks sends one message carrying half the book, so the hop size is $\Theta(1/p)$ of the corpus and this is the largest it will ever be in the series — 3,911 tokens here against 3,919 for the root's first binomial hop at $p = 8$, which also carries half the book. The reported `tokens_deferred` of 5,321 is the sum of the two rendezvous sends, and it disagrees with the `job.finish` event's 13,299 for the reason the $p = 8$ analysis identifies: `RankCost.tokens_deferred` is incremented both when a send chooses rendezvous and when a receive declines to admit, and this harness never admits anything. The value the analysis pipeline uses, 5,321, is the correct one.

What this point contributes to the series

Wall times are 576.74, 410.11, 308.44 and 166.78 s at $p = 1, 2, 4, 8$, so this point's speedup is $576.74/410.11 = 1.406\times$ at efficiency 0.703 and a Karp–Flatt serial fraction of 0.422 — the largest in the column, which then falls to 0.380 and 0.188. A falling Karp–Flatt is not the signature of a constant serial section, and the paper's USL fit of $\sigma = 0.220$, $\kappa = 0.0000$ is a compromise between this point's 0.422 and $p = 8$'s 0.188 that matches neither; its residuals have the sign pattern $-, -, +$ rather than a scatter.

This run supplies the cleanest single reason why the empirical serial fraction starts high and falls. Efficiency factorises exactly as $E(p) = (T_1/W(1)) \cdot (W(1)/W(p)) \cdot E_{\text{par}}(p)$, where $W(p)$ is model rank-seconds and E_{par} is the achieved parallel efficiency in the headline table. At this point that is $1.1131 \times 0.7728 \times 0.8175 = 0.7031$, against the table's 0.70. The middle term is the work amplification: $W(2) = 670.53$ rank-seconds against the baseline's 518.16, a factor of 1.294, of which 1.085 is the two extra revision calls and 1.193 is the sixteen shared calls simply taking longer — terminology 170.02 rank-seconds against 156.76 and translation 392.17 against 361.40, on byte-identical terminology prompts and translation prompts differing by five lines in 302. Dividing the amplification out of the whole series gives work-normalised speedups of 1.000, 1.820, 3.270 and 6.895 and a USL fit of $\sigma = 0.025$, $\kappa = 0.0000$, $R^2 = 0.989$.

Against its immediate neighbours, this point is where coordination is cheapest and least informative. Corrected coordination share is 10.0% here, 21.5% at $p = 4$ and 19.0% at $p = 8$. The run has one seam where $p = 8$ has seven, one collective peer where $p = 8$ has three tree neighbours and two

ring neighbours, and no distinction between a neighbourhood collective and a barrier at all. It establishes that the machinery works and how much it costs when there is almost nothing for it to do; the structural claims about the decomposition need the larger points.

8 Threats to this reading

The revision result rests on a diff of two blobs, and the diff is of text, not of meaning. That `revise:u3` returned its input byte for byte is a measurement and is not in doubt. That the revision phase "changed nothing" is a claim about the artefact, and the artefact is Chinese prose whose only recorded property here is its bytes. If the agents were right that the seam already read continuously, then returning the input unchanged is the correct behaviour and the phase did its job by confirming a negative — which is a service, just not one worth 19.2% of the rank-seconds. The trace cannot distinguish a correct no-op from a lazy one, and I have not read the Chinese.

Two ranks is a sample of two, and half the interesting numbers are the larger of two draws. The 22.245 s glossary wait, the 48.766 s halo wait and the 10.942 s barrier wait are all the difference between rank 1's completion and rank 0's, and rank 1 happened to be slower at every phase. A run in which the two ranks drew closer latencies would show the same structure with much smaller gaps and a coordination share nearer zero, without anything about the protocol having changed. The claim that rank 1 was slower *because* it holds 89 of the book's 149 paragraphs is an inference; it is supported by the pooled $r = +0.489$ between paragraphs and duration across 32 calls, but at $p = 2$ the per-rank evidence is two points.

The corrected coordination figure is a partition of intervals, not of causes. I attribute each idle interval to the collective whose event terminates it, which is well defined, but "rank 0 was blocked in `halo_exchange` for 48.766 s" and "rank 0 was blocked because rank 1's fourth translation ran long" are different kinds of statement. The first is measurement; the second is inference, well supported by rank 1's `translate:u7` completing at 347.40 s and rank 0 exiting the exchange 0.25 s later, but inference. The 61.4 rank-seconds of broker dispatch in the same table are not protocol at all and should not be read as coordination cost.

This is a broker-mediated run and the log cannot see inside a worker. `metrics.json` reports `executors: {broker: 2, worker: 33}`, and the outputs are real prose, so this is not a surrogate executor. But every one of the eighteen recorded latencies lies within 25 ms of a multiple of 0.5 s, which is what a scripted executor would also produce; I read it as the broker's `poll_s = 0.5` completion poll because that is its value in the source and no observed observation lag exceeds it.

Four of eight output sections are byte-identical to the sequential baseline's, and the pool was shared. The campaign ran in descending order of p, so this run preceded `real-tr-p1-full` by nine minutes against the same worker pool, and sections 0 through 3 — rank 0's whole allocation — are identical in the two runs. Both explanations must be on the table, and the campaign's usual verdict is convergence: with a binding 207-term glossary, a pinned paragraph count and an agreed register, a near-deterministic decoder has little room, so byte-identity is a short step rather than a leap. That argument is testable in this series and it does not survive the test.

The terminology phase is the instrument, because `prompt_terms` does not take the glossary and so a `terms:u`*n* prompt is a pure function of the unit. All four points of the series issue byte-identical terms prompts, 8 of 8 digests matching. If the executor were near-deterministic, all 32 results would agree unit by unit; instead only 6 of the 48 cross-run pairs do. `terms:u4` carries prompt

digest `ea002f41` everywhere and returns 36, 38, 38 and 34 terms in 913, 956, 956 and 872 bytes at $p = 1, 2, 4, 8$, with four differently worded register sentences. Identical input does not give identical output here, so determinism cannot explain a 1,300-character coincidence.

What the identities track instead is rank placement and adjacency in time. Over the four runs' 48 unit-pairs of final output, all 11 pairs in which the unit sat on rank 0 in both runs are byte-identical and none of the other 37 are; this run's rank-0 terms results are reproduced in full by $p = 1$ nine minutes later and match $p = 4$'s on only one of two units, while $p = 4$ matches none of $p = 8$'s. That is what reuse confined to one pool slot and decaying with distance looks like. Two things stop it being proof: a contiguous split puts the lowest-numbered units on rank 0 in every configuration, so "same rank" and "early unit" cannot be separated here, and the shared answers were generated rather than served — this run's `translate:u0` reports 44.52 s for its 403 output tokens, in line with the rest, where a cache hit would have returned at once. My reading is dependence between runs rather than convergence. It does not touch the timings, message counts or collective structure this document rests on, but it does mean the quality column of the scaling table is not four independent draws.

This point uses a code path the rest of the curve does not. Reduce and broadcast run `flat` here and `binomial` at $p = 4$ and $p = 8$, because the selector is `p <= 3`. Nothing in this document depends on the distinction, but any statement of the form "the glossary allreduce costs X as a function of p" drawn across all four points is fitting across an implementation change.